C++ TEXT EDITOR
FUNCTION-BY-FUNCTION GUIDE

Based on the /src source code

Last edited-27/10/2027

A practical explanation of the functions, data structures, control flow, and important implementation details.

How to use this guide

This document follows the actual files in /src folder. It explains what each function is responsible for, how it works at a high level, and which other parts of the editor it depends on. The goal is understanding, not merely memorizing definitions. Code is grouped by source file so you can keep the guide beside VS Code.

Important: this is a source-based explanation of the uploaded version, not a claim that every design choice is ideal. A few implementation caveats are called out explicitly where the code warrants them.

Core concepts to keep in mind

- A CharNode stores one character and links to the previous and next character.
- A LineNode represents one document line and owns a doubly linked list of CharNodes.
- A SkipNode is an auxiliary index pointing to a LineNode, used for line-number lookup.
- The cursor points to the character immediately to the right of the insertion point; NULL means end-of-line.
- Line numbers and cursor columns are zero-based internally.
- The editor converts its linked representation to std::string for saving and undo/redo snapshots.
- The Windows console APIs are used for low-level key input, rendering, and the system clipboard.

File map

- **main.cpp —** Starts the application and displays the final document after exit.
- **EditorNodes.h —** Defines character, line, and skip-list node structures.
- **TextEditor.h —** Declares TextEditor state and its public/private methods.
- **TextEditor.cpp —** Constructor, destructor, editing operations, and undo/redo coordination.
- **Cursor.cpp —** Cursor position calculations and arrow-key movement.
- **SkipList.cpp —** Builds, clears, and searches the line index.
- **Document.cpp —** Filename handling, open/save coordination, and string/document conversion.
- **Clipboard.cpp —** Selection, internal clipboard, Windows clipboard, copy/cut/paste.
- **InputHandler.cpp —** Reads Windows console events and dispatches keyboard commands.
- **Display.cpp —** Clears and renders the console, including selection highlighting.
- **Command.h / command.cpp —** Snapshot structure and undo/redo stacks.
- **FileHandler.h / FileHandler.cpp —** Low-level file read/write helpers.
- **ConsoleHelpers.h / ConsoleHelpers.cpp —** Console cursor positioning and Ctrl+C event handling.
- **build.bat —** Build helper for compiling the project (not a C++ function).
- **customWindow.svg —** Static visual asset; not part of the C++ function flow.
- **editor.exe —** Prebuilt executable; generated output, not source code.

Part I — Data structures and class state

CharNode (EditorNodes.h)

Fields: data stores one char; prev and next link to adjacent characters. CharNode(char ch) copies ch into data and initializes both links to NULL. The caller later links the node into a line.

LineNode (EditorNodes.h)

Fields: head and tail point to the first and last character; prev and next connect the line to neighbouring lines. LineNode() initializes all four pointers to NULL, so a new node starts empty and unlinked.

SkipNode and MAX_LEVEL (EditorNodes.h)

MAX_LEVEL is 4. Each SkipNode stores a LineNode pointer, a zero-based lineNumber, and four forward pointers. Its constructor stores the line and number and initializes all forward links to NULL.

EditorSnapshot and Command (Command.h)

EditorSnapshot stores the full document text plus the cursor's line and column. Command owns undoStack and redoStack, both stacks of snapshots. It separates history storage from the editing operations themselves.

TextEditor members (TextEditor.h)

- Document pointers: firstLine, lastLine, currentLine.
- Cursor state: cursor and currentLineNumber.
- Index state: skipHead and skipLevel.
- Selection state: hasSelection, anchorLine, anchorCol.
- Clipboard state: clipboard string plus clipHead, clipTail, clipLineCount for the internal linked-list copy.
- File state: currentFilename.
- History state: Command history and typingGroupOpen.
- Private helpers manipulate internal lists/indexes; public methods form the editor interface used by main() and start().

Part II — Every function, by source file

Each entry gives the purpose and the actual operational behaviour. Signatures are included to help you find the function quickly in the project.

1. Program entry point

main — Program entry point

Signature: int main()

Creates the TextEditor object, starts the interactive editor, clears the console after the editor exits, prints a heading, displays the final document, and returns success. The local object is constructed before start() and destroyed when main() exits.

```
int main()

{

    TextEditor editor;



    editor.start();



    system("cls");



    cout << "Final Text:\n\n";

    editor.showList();



    return 0;

}
```

2. Core data structures

The three node constructors and their fields are explained in Part I above.

3. TextEditor class interface

This file is primarily declarations rather than implementations. Read the method list as the class API; the implementation is explained in the corresponding .cpp sections.

- clearClipList — declared here; detailed implementation is explained in its source-file section.
- appendChar — declared here; detailed implementation is explained in its source-file section.
- buildClipList — declared here; detailed implementation is explained in its source-file section.
- cloneChain — declared here; detailed implementation is explained in its source-file section.
- insertChainAtCursor — declared here; detailed implementation is explained in its source-file section.
- pasteClipList — declared here; detailed implementation is explained in its source-file section.
- clearSkipList — declared here; detailed implementation is explained in its source-file section.
- rebuildSkipList — declared here; detailed implementation is explained in its source-file section.
- getLine — declared here; detailed implementation is explained in its source-file section.
- getCurrentColumn — declared here; detailed implementation is explained in its source-file section.
- getLineLength — declared here; detailed implementation is explained in its source-file section.
- setCursorColumn — declared here; detailed implementation is explained in its source-file section.
- updateCurrentLineNumber — declared here; detailed implementation is explained in its source-file section.
- TextEditor — declared here; detailed implementation is explained in its source-file section.
- normalizeFilename — declared here; detailed implementation is explained in its source-file section.
- readFilename — declared here; detailed implementation is explained in its source-file section.
- saveDocument — declared here; detailed implementation is explained in its source-file section.
- openDocument — declared here; detailed implementation is explained in its source-file section.
- getText — declared here; detailed implementation is explained in its source-file section.
- setText — declared here; detailed implementation is explained in its source-file section.
- moveTo — declared here; detailed implementation is explained in its source-file section.
- getSelectionRange — declared here; detailed implementation is explained in its source-file section.
- selectionActive — declared here; detailed implementation is explained in its source-file section.
- beginSelection — declared here; detailed implementation is explained in its source-file section.
- clearSelection — declared here; detailed implementation is explained in its source-file section.
- selectAll — declared here; detailed implementation is explained in its source-file section.
- textBetween — declared here; detailed implementation is explained in its source-file section.
- deleteSelection — declared here; detailed implementation is explained in its source-file section.
- setSystemClipboard — declared here; detailed implementation is explained in its source-file section.
- getSystemClipboard — declared here; detailed implementation is explained in its source-file section.
- copy — declared here; detailed implementation is explained in its source-file section.
- cut — declared here; detailed implementation is explained in its source-file section.
- paste — declared here; detailed implementation is explained in its source-file section.
- insert — declared here; detailed implementation is explained in its source-file section.
- moveLeft — declared here; detailed implementation is explained in its source-file section.
- moveRight — declared here; detailed implementation is explained in its source-file section.
- backspace — declared here; detailed implementation is explained in its source-file section.
- deleteChar — declared here; detailed implementation is explained in its source-file section.
- mergeWithPreviousLine — declared here; detailed implementation is explained in its source-file section.
- mergeWithNextLine — declared here; detailed implementation is explained in its source-file section.
- newLine — declared here; detailed implementation is explained in its source-file section.
- moveUp — declared here; detailed implementation is explained in its source-file section.
- moveDown — declared here; detailed implementation is explained in its source-file section.
- flushBuffer — declared here; detailed implementation is explained in its source-file section.
- clearScreen — declared here; detailed implementation is explained in its source-file section.
- display — declared here; detailed implementation is explained in its source-file section.
- start — declared here; detailed implementation is explained in its source-file section.
- showList — declared here; detailed implementation is explained in its source-file section.

- captureSnapshot — declared here; detailed implementation is explained in its source-file section.
- recordEdit — declared here; detailed implementation is explained in its source-file section.
- undo — declared here; detailed implementation is explained in its source-file section.
- redo — declared here; detailed implementation is explained in its source-file section.

4. Core editing and history functions

TextEditor::TextEditor — Constructor

Signature: TextEditor::TextEditor()

Initializes the editor's state: creates the first empty line, sets current/first/last line pointers, initializes cursor and line number, initializes selection and clipboard fields, sets up typing-history state, and builds the skip-list index. It also seeds the pseudo-random generator, although the shown skip-list builder does not use randomness.

```
TextEditor::TextEditor()
{
    srand((unsigned)time(NULL));
    currentFilename = "";

    firstLine = new LineNode();

    lastLine = firstLine;
    currentLine = firstLine;

    cursor = NULL;

    currentLineNumber = 0;

    skipHead = NULL;
    skipLevel = 1;

    typingGroupOpen = false;
    hasSelection = false;
    anchorLine = 0;
    anchorCol = 0;

    clipHead = NULL;
    clipTail = NULL;
    clipLineCount = 0;

    rebuildSkipList();
}
```

TextEditor::insert — Insert one character

Signature: void TextEditor::insert(char ch)

Allocates a CharNode for the character and inserts it at the current cursor position. It handles an empty line, insertion at end-of-line (cursor == NULL), and insertion before an existing character. It updates neighbouring links and the line head/tail as needed. Cursor convention: cursor points to the character immediately to the right of the insertion point, or NULL at end-of-line.

```
void TextEditor::insert(char ch)
{
        CharNode* newNode = new CharNode(ch);

        if (currentLine->head == NULL)
        {
            currentLine->head = newNode;
            currentLine->tail = newNode;

            cursor = NULL;

            return;
        }

        if (cursor == NULL)
        {
```

```
            newNode->prev = currentLine->tail;

            currentLine->tail->next =
                newNode;

            currentLine->tail =
                newNode;

            return;
        }

        CharNode* left = cursor->prev;

        newNode->next = cursor;
        newNode->prev = left;

        cursor->prev = newNode;

        if (left != NULL)
        {
            left->next = newNode;
        }
        else
        {
            currentLine->head = newNode;
        }
    }
```

TextEditor::backspace — Delete character before cursor

Signature: void TextEditor::backspace()

Removes the character immediately to the left of the cursor. At the start of a line, it joins the current line to the previous line; at end-of-line, it removes the tail character. It repairs prev/next links and head/tail pointers before deleting the node.

```
void TextEditor::backspace()
{
        if (currentLine->head == NULL)
        {
            if (currentLine->prev != NULL)
            {
                mergeWithPreviousLine();
            }

            return;
        }

        if (cursor == currentLine->head)
        {
            if (currentLine->prev != NULL)
            {
                mergeWithPreviousLine();
            }

            return;
        }

        if (cursor == NULL)
        {
            CharNode* del =
                currentLine->tail;

            currentLine->tail =
                del->prev;

            if (currentLine->tail != NULL)
            {
                currentLine->tail->next =
                    NULL;
            }
            else
            {
                currentLine->head = NULL;
```

```
            }

            delete del;

            return;
        }

        CharNode* del =
            cursor->prev;

        if (del == NULL)
            return;

        CharNode* left =
            del->prev;

        cursor->prev = left;

        if (left != NULL)
        {
            left->next = cursor;
        }
        else
        {
            currentLine->head = cursor;
        }

        delete del;
    }
```

TextEditor::deleteChar — Delete character at cursor

Signature: void TextEditor::deleteChar()

Deletes the character currently pointed to by cursor. If cursor is NULL and a next line exists, it merges that next line into the current one, effectively deleting the newline. The cursor advances to the next character after deletion.

```
void TextEditor::deleteChar()
{
        if (cursor == NULL)
        {
            if (currentLine->next != NULL)
            {
                mergeWithNextLine();
            }

            return;
        }

        CharNode* del = cursor;

        cursor = cursor->next;

        if (del->prev != NULL)
        {
            del->prev->next = cursor;
        }
        else
        {
            currentLine->head = cursor;
        }

        if (cursor != NULL)
        {
            cursor->prev = del->prev;
        }
        else
        {
            currentLine->tail =
                del->prev;
        }

        delete del;
    }
```

TextEditor::mergeWithPreviousLine — Join current line to previous line

Signature: void TextEditor::mergeWithPreviousLine()

Moves the current line's characters onto the end of the previous line, unlinks and deletes the current LineNode, updates lastLine if necessary, moves currentLine to the previous line, restores the cursor at the join point, recalculates the line number, and rebuilds the skip list. It has a separate path for an empty current line.

```
void TextEditor::mergeWithPreviousLine()
{
        LineNode* previous = currentLine->prev;

        if (previous == NULL)
            return;

        CharNode* firstOfCurrent = currentLine->head;

        if (currentLine->head == NULL)
        {
            previous->next = currentLine->next;

            if (currentLine->next != NULL)
            {
                currentLine->next->prev = previous;
            }
            else
            {
                lastLine = previous;
            }

            delete currentLine;

            currentLine = previous;

            cursor = NULL;

            updateCurrentLineNumber();

            rebuildSkipList();

            return;
        }

        if (previous->head == NULL)
        {
            previous->head = currentLine->head;
            previous->tail = currentLine->tail;
        }
        else
        {
            previous->tail->next = currentLine->head;
            currentLine->head->prev = previous->tail;
            previous->tail = currentLine->tail;
        }

        previous->next = currentLine->next;

        if (currentLine->next != NULL)
        {
            currentLine->next->prev = previous;
        }
        else
        {
            lastLine = previous;
        }

        delete currentLine;

        currentLine = previous;

        cursor = firstOfCurrent;

        updateCurrentLineNumber();
```

```
        rebuildSkipList();
    }
```

TextEditor::mergeWithNextLine — Join next line to current line

Signature: void TextEditor::mergeWithNextLine()

Appends the next line's character chain to the current line, unlinks and deletes the next LineNode, updates the document links and lastLine, places the cursor at the join point, and rebuilds the skip list.

```
void TextEditor::mergeWithNextLine()
{
        LineNode* nextLine =
            currentLine->next;

        if (nextLine == NULL)
            return;

        // Cursor ends up at the join point: just before the first
        // character that came from the next line (NULL if none).
        CharNode* joinPoint = nextLine->head;

        if (currentLine->head == NULL)
        {
            currentLine->head =
                nextLine->head;

            currentLine->tail =
                nextLine->tail;
        }
        else if (nextLine->head != NULL)
        {
            currentLine->tail->next =
                nextLine->head;

            nextLine->head->prev =
                currentLine->tail;

            currentLine->tail =
                nextLine->tail;
        }

        currentLine->next =
            nextLine->next;

        if (nextLine->next != NULL)
        {
            nextLine->next->prev =
                currentLine;
        }
        else
        {
            lastLine = currentLine;
        }

        delete nextLine;

        cursor = joinPoint;

        rebuildSkipList();
    }
```

TextEditor::newLine — Split line at cursor

Signature: void TextEditor::newLine()

Creates a new LineNode after the current line. If cursor is at end-of-line, it inserts an empty line after the current line. Otherwise, it moves the cursor-and-right-side character chain into the new line, leaving the left side in the old line. It updates line links, currentLine, cursor, line number, and skip-list index.

```
void TextEditor::newLine()
{
    LineNode* newLine =
        new LineNode();
```

```
if (cursor == NULL)
{
    newLine->prev =
        currentLine;

    newLine->next =
        currentLine->next;

    if (currentLine->next != NULL)
    {
        currentLine->next->prev =
            newLine;
    }
    else
    {
        lastLine = newLine;
    }

    currentLine->next =
        newLine;

    currentLine =
        newLine;

    cursor = NULL;

    updateCurrentLineNumber();

    rebuildSkipList();

    return;
}

newLine->head = cursor;
newLine->tail =
    currentLine->tail;

CharNode* left =
    cursor->prev;

if (left != NULL)
{
    left->next = NULL;

    currentLine->tail =
        left;
}
else
{
    currentLine->head = NULL;
    currentLine->tail = NULL;
}

cursor->prev = NULL;

newLine->prev =
    currentLine;

newLine->next =
    currentLine->next;

if (currentLine->next != NULL)
{
    currentLine->next->prev =
        newLine;
}
else
{
    lastLine = newLine;
}

currentLine->next =
    newLine;
```

```
    currentLine =
        newLine;

    cursor =
        newLine->head;

    updateCurrentLineNumber();

    rebuildSkipList();
}
```

TextEditor::captureSnapshot — Capture undo/redo state

Signature: EditorSnapshot TextEditor::captureSnapshot()

Creates an EditorSnapshot containing the current document as a string, current line number, and current column. The history system uses this snapshot to restore text and cursor position.

```
EditorSnapshot TextEditor::captureSnapshot()
{
    EditorSnapshot snapshot;
    snapshot.text = getText();
    snapshot.line = currentLineNumber;
    snapshot.column = getCurrentColumn();
    return snapshot;
}
```

TextEditor::recordEdit — Record an edit for history

Signature: void TextEditor::recordEdit(const EditorSnapshot& before,bool isTyping,bool endGroup)

Captures the post-edit snapshot and passes states to Command history. Typing edits are grouped: only the first edit in an open typing group is recorded, and endGroup closes that group. Non-typing edits close the typing group and record a history state. Command::record ignores changes whose text is unchanged.

History note: the first character of a typing group records the pre-edit snapshot; later characters in the same group are not individually pushed. A space closes the group when start() passes endGroup=true.

```
void TextEditor::recordEdit(const EditorSnapshot& before,bool isTyping,bool endGroup)
{
    EditorSnapshot after = captureSnapshot();

    if (isTyping)
    {
        if (!typingGroupOpen)
        {
            history.record(before, after);
            typingGroupOpen = true;
        }

        if (endGroup)
            typingGroupOpen = false;
    }
    else
    {
        typingGroupOpen = false;
        history.record(before, after);
    }
}
```

TextEditor::undo — Undo most recent recorded change

Signature: void TextEditor::undo()

Closes any typing group, asks history for the previous snapshot, and if one exists replaces the document with that snapshot's text, restores the cursor line/column, and clears selection.

```
void TextEditor::undo()
{
    typingGroupOpen = false;

    EditorSnapshot target;
    if (history.undo(captureSnapshot(), target))
```

```
    {
        setText(target.text);
        moveTo(target.line, target.column);
        clearSelection();
    }
}
```

TextEditor::redo — Redo a previously undone change

Signature: void TextEditor::redo()

Closes any typing group, asks history for the next snapshot, and if one exists restores its text and cursor position and clears selection.

```
void TextEditor::redo()
{
    typingGroupOpen = false;

    EditorSnapshot target;
    if (history.redo(captureSnapshot(), target))
    {
        setText(target.text);
        moveTo(target.line, target.column);
        clearSelection();
    }
}
```

5. Cursor movement and position helpers

TextEditor::getCurrentColumn — Calculate cursor column

Signature: int TextEditor::getCurrentColumn()

Walks the current line from its head, counting characters until it reaches cursor. If cursor is NULL or is not found in the chain, the count reaches the line length, representing end-of-line.

```
int TextEditor::getCurrentColumn()

{

    int column = 0;



    CharNode* temp = currentLine->head;



    while (temp != NULL)

    {

        if (temp == cursor)

            return column;



        column++;



        temp = temp->next;

    }



    return column;

}
```

TextEditor::getLineLength — Count characters in a line

Signature: int TextEditor::getLineLength(LineNode* line)

Traverses a LineNode's character list and returns the number of character nodes.

```
int TextEditor::getLineLength(LineNode* line)

{

    int length = 0;



    CharNode* temp = line->head;



    while (temp != NULL)

    {

        length++;



        temp = temp->next;

    }



    return length;

}
```

TextEditor::setCursorColumn — Move cursor to column

Signature: void TextEditor::setCursorColumn(int column)

Starts at the line head and advances by the requested number of characters. A non-positive column places cursor at the head; a column beyond the end naturally results in cursor == NULL, the end-of-line position.

```
void TextEditor::setCursorColumn(int column)

{

    if (column <= 0)

    {

        cursor = currentLine->head;

        return;

    }



    CharNode* temp = currentLine->head;



    int currentColumn = 0;



    while (

        temp != NULL &&

        currentColumn < column

    )
```

```
    {
        temp = temp->next;
        currentColumn++;
    }

    cursor = temp;
}
```

TextEditor::updateCurrentLineNumber — Recalculate current line index

Signature: void TextEditor::updateCurrentLineNumber()

Walks from firstLine and counts lines until it finds currentLine. The counter is zero-based. If currentLine is not reachable from firstLine, the function finishes with the count of traversed lines rather than reporting an explicit error.

```
void TextEditor::updateCurrentLineNumber()
{
    currentLineNumber = 0;

    LineNode* temp = firstLine;

    while (temp != NULL)
    {
        if (temp == currentLine)
            return;

        currentLineNumber++;

        temp = temp->next;
    }
}
```

TextEditor::moveTo — Move to line and column

Signature: void TextEditor::moveTo(int line, int column)

Walks forward from firstLine until the requested line index is reached or the last line is reached. It then updates currentLine/currentLineNumber and calls setCursorColumn(), so oversized line and column values are effectively clamped to the document end.

```
void TextEditor::moveTo(int line, int column)
{
    LineNode* l = firstLine;
    int n = 0;

    while (n < line && l->next != NULL)
    {
```

```
        l = l->next;
        n++;
    }

    currentLine = l;
    currentLineNumber = n;

    setCursorColumn(column);
}
```

TextEditor::moveLeft — Move cursor left

Signature: void TextEditor::moveLeft()

Moves cursor to the preceding character when possible. With the cursor at end-of-line (NULL), it moves to the line's tail. At the beginning of a line, it can move to the previous line's end. It updates the current line number when crossing lines.

```
void TextEditor::moveLeft()
{
    if (cursor != NULL)
    {
        if (cursor->prev != NULL)
        {
            cursor = cursor->prev;
        }
        else
        {
            if (currentLine->prev != NULL)
            {
                currentLine =
                    currentLine->prev;

                updateCurrentLineNumber();

                cursor = NULL;
            }
        }

        return;
    }
```

```
    if (currentLine->tail != NULL)
    {
        cursor = currentLine->tail;
    }
    else
    {
        if (currentLine->prev != NULL)
        {
            currentLine =
                currentLine->prev;

            updateCurrentLineNumber();

            cursor = NULL;
        }
    }
}
```

TextEditor::moveRight — Move cursor right

Signature: void TextEditor::moveRight()

Advances cursor to the next character. At the end of a line it crosses to the next line's head when one exists; at the end of the final line it sets cursor to NULL. It updates the current line number when crossing lines.

```
void TextEditor::moveRight()
{
    if (cursor != NULL)
    {
        if (cursor->next != NULL)
        {
            cursor = cursor->next;
        }
        else
        {
            if (currentLine->next != NULL)
            {
                currentLine =
                    currentLine->next;

                updateCurrentLineNumber();
```

```
            cursor =
                currentLine->head;
        }
        else
        {
            cursor = NULL;
        }
    }

    return;
}

if (currentLine->next != NULL)
{
    currentLine =
        currentLine->next;

    updateCurrentLineNumber();

    cursor =
        currentLine->head;
}
}
```

TextEditor::moveUp — Move cursor up one line

Signature: void TextEditor::moveUp()

Does nothing on the first line. Otherwise it remembers the current column, finds the preceding line via getLine(), switches to it, and clamps the column to that line's length.

Cursor note: vertical movement attempts to preserve the current column but clamps it when the destination line is shorter.

```
void TextEditor::moveUp()
{
    if (currentLineNumber == 0)
        return;

    int column =
        getCurrentColumn();

    int targetLineNumber =
```

```
        currentLineNumber - 1;

    LineNode* target =
        getLine(targetLineNumber);

    if (target == NULL)
        return;

    currentLine = target;

    currentLineNumber =
        targetLineNumber;

    int length =
        getLineLength(currentLine);

    column =
        min(column, length);

    setCursorColumn(column);
}
```

TextEditor::moveDown — Move cursor down one line

Signature: void TextEditor::moveDown()

Remembers the current column, looks up the following line through getLine(), and moves to the same column where possible. If the target line is shorter, the cursor is clamped to its end; if no target line exists, nothing changes.

Cursor note: vertical movement attempts to preserve the current column but clamps it when the destination line is shorter.

```
void TextEditor::moveDown()
{
    int column =
        getCurrentColumn();

    int targetLineNumber =
        currentLineNumber + 1;

    LineNode* target =
        getLine(targetLineNumber);
```

```
    if (target == NULL)
        return;

    currentLine = target;

    currentLineNumber =
        targetLineNumber;

    int length =
        getLineLength(currentLine);

    column =
        min(column, length);

    setCursorColumn(column);
}
```

6. Skip-list indexing

TextEditor::clearSkipList — Free skip-list index nodes

Signature: void TextEditor::clearSkipList()

Walks the skip-list bottom level using forward[0], deletes each SkipNode, then resets skipHead to NULL and skipLevel to 1. It deletes index nodes, not the LineNodes referenced by them.

```
void TextEditor::clearSkipList()
{
    if (skipHead == NULL)
        return;

    SkipNode* current = skipHead;

    while (current != NULL)
    {
        SkipNode* next = current->forward[0];

        delete current;

        current = next;
    }
```

```
    skipHead = NULL;

    skipLevel = 1;

}
```

TextEditor::rebuildSkipList — Rebuild skip-list index

Signature: void TextEditor::rebuildSkipList()

Clears the old index, collects all document LineNode pointers into a vector, creates one SkipNode per line, connects level 0 to the next node, and connects higher levels at power-of-two intervals. The first skip node becomes skipHead. The current implementation sets skipLevel to MAX_LEVEL even when the document has fewer lines.

Complexity note: rebuilding the index traverses all lines and recreates all SkipNodes. It is straightforward but costs O(number of lines) per rebuild.

```
void TextEditor::rebuildSkipList()

{

    clearSkipList();



    vector<LineNode*> lines;



    LineNode* line = firstLine;



    while (line != NULL)

    {

        lines.push_back(line);

        line = line->next;

    }



    if (lines.empty())

        return;



    vector<SkipNode*> nodes;



    for (int i = 0; i < (int)lines.size(); i++)

    {

        nodes.push_back(

            new SkipNode(lines[i], i)

        );

    }



    for (int i = 0; i < (int)nodes.size() - 1; i++)
```

```
    {
        nodes[i]->forward[0] =
            nodes[i + 1];
    }

    for (int level = 1;
            level < MAX_LEVEL;
            level++)
    {
        int step = 1 << level;

        for (int i = 0;
                i + step < (int)nodes.size();
                i += step)
        {
            nodes[i]->forward[level] =
                nodes[i + step];
        }
    }

    skipHead = nodes[0];

    skipLevel = MAX_LEVEL;
}
```

TextEditor::getLine — Look up line by zero-based index

Signature: LineNode* TextEditor::getLine(int lineNumber)

Uses the skip list's forward pointers from the highest level down to find the last indexed line whose number does not exceed the requested index. Returns the matching LineNode or NULL for invalid/unavailable line numbers.

```
LineNode* TextEditor::getLine(int lineNumber)
{
    if (skipHead == NULL)
        return NULL;

    if (lineNumber < 0)
        return NULL;

    SkipNode* current = skipHead;
```

```
    for (int level = MAX_LEVEL - 1;
            level >= 0;
            level--)
    {
        while (
            current->forward[level] != NULL &&
            current->forward[level]->lineNumber <= lineNumber
        )
        {
            current =
                current->forward[level];
        }
    }

    if (current->lineNumber == lineNumber)
        return current->line;

    return NULL;
}
```

7. Document conversion and file operations

TextEditor::normalizeFilename — Add default file extension

Signature: string TextEditor::normalizeFilename(string filename)

Returns an empty filename unchanged. Otherwise it checks the last slash and last dot; if there is no extension after the final path separator, it appends .txt. Dots in directory names are not treated as file extensions.

```
string TextEditor::normalizeFilename(string filename)
{
        if (filename.empty())
            return filename;

        // Only inspect the filename, not dots in the directory path.
        size_t slash = filename.find_last_of("\\/");
        size_t dot = filename.find_last_of('.');

        if (dot == string::npos ||
            (slash != string::npos && dot < slash))
        {
```

```
            filename += ".txt";
        }

        return filename;
    }
```

TextEditor::readFilename — Read a filename from console input

Signature: string TextEditor::readFilename()

Reads low-level Windows console key events so it can handle Enter, Escape, Backspace, and printable ASCII characters. Enter accepts the typed filename; Escape cancels by clearing it. Ctrl-modified printable input is ignored.

```
string TextEditor::readFilename()
{
        HANDLE hInput = GetStdHandle(STD_INPUT_HANDLE);
        HANDLE hOutput = GetStdHandle(STD_OUTPUT_HANDLE);

        INPUT_RECORD input;
        DWORD events;

        string filename;

        while (true)
        {
            if (!ReadConsoleInput(hInput, &input, 1, &events))
                break;

            if (input.EventType != KEY_EVENT)
                continue;

            KEY_EVENT_RECORD key = input.Event.KeyEvent;

            if (!key.bKeyDown)
                continue;

            WORD vk = key.wVirtualKeyCode;
            char ch = key.uChar.AsciiChar;

            if (vk == VK_RETURN)
            {
```

```
            cout << '\n';
            break;
        }

        if (vk == VK_ESCAPE)
        {
            filename.clear();
            cout << '\n';
            break;
        }

        if (vk == VK_BACK)
        {
            if (!filename.empty())
            {
                filename.pop_back();
                cout << "\b \b";
            }

            continue;
        }

        bool ctrl = (key.dwControlKeyState &
            (LEFT_CTRL_PRESSED | RIGHT_CTRL_PRESSED)) != 0;

        if (!ctrl && ch >= 32 && ch <= 126)
        {
            filename += ch;
            cout << ch;
        }
    }

    return filename;
}
```

TextEditor::openDocument — Open a document

Signature: bool TextEditor::openDocument()

Prompts for a filename, reads it with readFilename(), normalizes the extension, loads file contents through loadFromFile(), rebuilds the editor document with setText(), and stores currentFilename only after a successful load. Returns false on cancellation or failure.

```
bool TextEditor::openDocument()

{

        cout << "\nEnter filename or full path to open: ";



        string filename = readFilename();



        if (filename.empty())

        {

            cout << "Open cancelled.\n";

            return false;

        }



        filename = normalizeFilename(filename);



        string content;



        if (!loadFromFile(filename, content))

        {

            cout << "\nCould not open file: "

                << filename << "\n";



            return false;

        }



        setText(content);



        currentFilename = filename;



        cout << "\nOpened successfully: "

            << currentFilename << "\n";



        return true;

    }
```

TextEditor::getText — Convert linked document to string

Signature: string TextEditor::getText()

Walks every LineNode and every CharNode, appending characters to a std::string. It inserts '\n' between line nodes, but not after the final line. This is used for saving and snapshot capture.

Serialization note: the string has newline separators between stored lines but no extra newline after the final line.

```
string TextEditor::getText()

{

        string content;


        LineNode* line = firstLine;


        while (line != NULL)

        {

            CharNode* ch = line->head;


            while (ch != NULL)

            {

                content += ch->data;

                ch = ch->next;

            }


            if (line->next != NULL)

                content += '\n';


            line = line->next;

        }


        return content;

    }
```

TextEditor::setText — Replace document contents

Signature: void TextEditor::setText(const string& content)

Deletes all existing line and character nodes, creates one fresh empty line, clears selection, and parses the supplied string into linked lines and characters. Newline characters create new LineNodes; tabs expand to four spaces; printable ASCII is stored. It resets the cursor to the start and rebuilds the skip list.

Data-flow note: setText() rebuilds the document from scratch. Since it deletes the previous line list, callers should ensure no other pointers into the old document are still expected to remain valid.

```
void TextEditor::setText(const string& content)

{

        // Delete all existing lines and characters.

        LineNode* line = firstLine;
```

```
while (line != NULL)
{
    CharNode* ch = line->head;

    while (ch != NULL)
    {
        CharNode* nextChar = ch->next;
        delete ch;
        ch = nextChar;
    }

    LineNode* nextLine = line->next;
    delete line;
    line = nextLine;
}

// Start with one empty line.
firstLine = new LineNode();
lastLine = firstLine;
currentLine = firstLine;
cursor = NULL;
currentLineNumber = 0;

clearSelection();

// Rebuild the linked list from the loaded string.
for (size_t i = 0; i < content.size(); ++i)
{
    char ch = content[i];

    if (ch == '\n')
    {
        LineNode* newNode = new LineNode();

        newNode->prev = lastLine;
        lastLine->next = newNode;
        lastLine = newNode;
```

```
            }
            else if (ch == '\t')
            {
                for (int k = 0; k < 4; ++k)
                    appendChar(lastLine, ' ');
            }
            else if (ch >= 32 && ch <= 126)
            {
                appendChar(lastLine, ch);
            }
        }

        // Place the cursor at the beginning.
        currentLine = firstLine;
        currentLineNumber = 0;
        cursor = firstLine->head;

        // Update line lookup after rebuilding the document.
        rebuildSkipList();
    }
```

TextEditor::saveDocument — Save current document or Save As

Signature: bool TextEditor::saveDocument(bool saveAs)

Uses currentFilename unless Save As was requested or no filename exists. It prompts for a name when needed, normalizes the extension, serializes the linked document with getText(), and calls saveToFile(). On success it updates currentFilename; otherwise it reports failure and returns false.

```
bool TextEditor::saveDocument(bool saveAs)
{
        string filename = currentFilename;

        if (saveAs || filename.empty())
        {
            cout << "\nEnter filename or full path: ";

            filename = readFilename();

            if (filename.empty())
            {
                cout << "Save cancelled.\n";
```

```
            return false;
        }

        filename = normalizeFilename(filename);
    }

    if (saveToFile(filename, getText()))
    {
        currentFilename = filename;

        cout << "\nSaved successfully: "
            << currentFilename << "\n";

        return true;
    }

    cout << "\nCould not save file: "
        << filename << "\n";

    return false;
}
```

8. Selection and clipboard operations

TextEditor::clearClipList — Free internal clipboard nodes

Signature: void TextEditor::clearClipList()

Traverses every clipboard LineNode and each of its CharNodes, deleting character nodes first and then the line node. Finally it resets clipboard head/tail pointers and line count.

```
void TextEditor::clearClipList()
{
    LineNode* line = clipHead;

    while (line != NULL)
    {
        CharNode* c = line->head;

        while (c != NULL)
        {
```

```
            CharNode* nextChar = c->next;
            delete c;
            c = nextChar;
        }

        LineNode* nextLine = line->next;
        delete line;
        line = nextLine;
    }

    clipHead = NULL;
    clipTail = NULL;
    clipLineCount = 0;
}
```

TextEditor::appendChar — Append character to a clipboard/document line

Signature: void TextEditor::appendChar(LineNode* line, char ch)

Allocates a CharNode and appends it to the supplied LineNode. If the line is empty, it sets both head and tail; otherwise it links after the current tail and updates tail.

```
void TextEditor::appendChar(LineNode* line, char ch)
{
    CharNode* n = new CharNode(ch);

    if (line->tail == NULL)
    {
        line->head = n;
        line->tail = n;
    }
    else
    {
        n->prev = line->tail;
        line->tail->next = n;
        line->tail = n;
    }
}
```

TextEditor::buildClipList — Build linked-list clipboard representation

Signature: void TextEditor::buildClipList(const string& text)

Clears the previous clipboard list, creates an initial line, then parses the text. Newlines create new clipboard LineNodes, tabs expand to four spaces, and printable ASCII characters are appended as character nodes. It updates clipLineCount.

```
void TextEditor::buildClipList(const string& text)
{
        clearClipList();

        LineNode* line = new LineNode();

        clipHead = line;
        clipTail = line;
        clipLineCount = 1;

        for (size_t i = 0; i < text.size(); i++)
        {
            char ch = text[i];

            if (ch == '\n')
            {
                LineNode* nl = new LineNode();

                nl->prev = clipTail;
                clipTail->next = nl;
                clipTail = nl;
                line = nl;

                clipLineCount++;
            }
            else if (ch == '\t')
            {
                for (int k = 0; k < 4; k++)
                    appendChar(line, ' ');
            }
            else if (ch >= 32 && ch <= 126)
            {
                appendChar(line, ch);
            }
        }
    }
```

TextEditor::cloneChain — Deep-copy a character chain

Signature: void TextEditor::cloneChain(CharNode* src, CharNode*& h, CharNode*& t)

Creates new CharNodes with the same character data as the source chain and links them into a separate doubly linked list. The output head and tail references are reset first. It does not alter the source.

```
void TextEditor::cloneChain(CharNode* src, CharNode*& h, CharNode*& t)

{

        h = NULL;

        t = NULL;



        for (; src != NULL; src = src->next)

        {

            CharNode* n = new CharNode(src->data);



            if (t == NULL)

            {

                h = n;

                t = n;

            }

            else

            {

                n->prev = t;

                t->next = n;

                t = n;

            }

        }

    }
```

TextEditor::insertChainAtCursor — Splice a character chain at cursor

Signature: void TextEditor::insertChainAtCursor(CharNode* h, CharNode* t)

Inserts a previously built chain into the current line. It handles an empty line, end-of-line insertion, and insertion before the cursor. It connects the chain's endpoints to the existing left and right neighbours and updates the line head/tail where needed.

```
void TextEditor::insertChainAtCursor(CharNode* h, CharNode* t)

{

        if (h == NULL)

            return;



        if (currentLine->head == NULL)

        {

            currentLine->head = h;
```

```
            currentLine->tail = t;
            cursor = NULL;
            return;
        }

        if (cursor == NULL)
        {
            h->prev = currentLine->tail;
            currentLine->tail->next = h;
            currentLine->tail = t;
            return;
        }

        CharNode* left = cursor->prev;

        h->prev = left;
        t->next = cursor;
        cursor->prev = t;

        if (left != NULL)
            left->next = h;
        else
            currentLine->head = h;
    }
```

TextEditor::pasteClipList — Paste internal clipboard list

Signature: void TextEditor::pasteClipList()

For a single-line clipboard, clones its character chain and splices it at the cursor. For multi-line content, splits the current line at the cursor, appends the first clipboard line to the left part, creates new document lines for remaining clipboard lines, attaches the original right-hand text after the pasted lines, updates line links/current position/line number, and rebuilds the skip list.

Pointer note: multi-line paste is a chain-splicing operation. Correctness depends on maintaining both directions of every line/character link and restoring the old right-hand text after the pasted block.

```
void TextEditor::pasteClipList()
{
        if (clipHead == NULL)
            return;

        CharNode* h;
```

```
CharNode* t;

// ---- single-line clipboard: one chain splice ----
if (clipLineCount == 1)
{
    cloneChain(clipHead->head, h, t);
    insertChainAtCursor(h, t);
    return;
}

// ---- multi-line clipboard ----
LineNode* cur = currentLine;

// 1. Cut the current line at the cursor; keep the right half aside.
CharNode* rightHead = cursor;
CharNode* rightTail = (cursor != NULL) ? cur->tail : NULL;

if (cursor != NULL)
{
    CharNode* left = cursor->prev;

    if (left != NULL)
    {
        left->next = NULL;
        cur->tail = left;
    }
    else
    {
        cur->head = NULL;
        cur->tail = NULL;
    }

    cursor->prev = NULL;
}

// 2. First clipboard line continues the current line.
cloneChain(clipHead->head, h, t);
```

```
if (h != NULL)
{
    if (cur->tail != NULL)
    {
        cur->tail->next = h;
        h->prev = cur->tail;
        cur->tail = t;
    }
    else
    {
        cur->head = h;
        cur->tail = t;
    }
}

// 3. Remaining clipboard lines become new lines after it.
LineNode* after = cur->next;
LineNode* last = cur;

for (LineNode* src = clipHead->next; src != NULL; src = src->next)
{
    LineNode* nl = new LineNode();

    cloneChain(src->head, nl->head, nl->tail);

    nl->prev = last;
    last->next = nl;
    last = nl;
}

// 4. Re-attach the right half after the last pasted line.
if (rightHead != NULL)
{
    if (last->tail != NULL)
    {
        last->tail->next = rightHead;
```

```
            rightHead->prev = last->tail;
            last->tail = rightTail;
        }
        else
        {
            last->head = rightHead;
            last->tail = rightTail;
        }
    }

    last->next = after;

    if (after != NULL)
        after->prev = last;
    else
        lastLine = last;

    currentLine = last;
    cursor = rightHead;
    currentLineNumber += clipLineCount - 1;

    rebuildSkipList();
}
```

TextEditor::getSelectionRange — Normalize selection endpoints

Signature: bool TextEditor::getSelectionRange(int& sl, int& sc, int& el, int& ec)

Returns false if no selection exists or the anchor and cursor positions are identical. Otherwise compares the anchor and current cursor positions and writes the earlier endpoint to (sl,sc) and later endpoint to (el,ec), regardless of selection direction.

```
bool TextEditor::getSelectionRange(int& sl, int& sc, int& el, int& ec)
{
    if (!hasSelection)
        return false;

    int curLine = currentLineNumber;
    int curCol = getCurrentColumn();

    if (curLine == anchorLine && curCol == anchorCol)
        return false;
```

```
        if (anchorLine < curLine ||
            (anchorLine == curLine && anchorCol < curCol))
        {
            sl = anchorLine; sc = anchorCol;
            el = curLine;    ec = curCol;
        }
        else
        {
            sl = curLine;    sc = curCol;
            el = anchorLine; ec = anchorCol;
        }

        return true;
    }
```

TextEditor::selectionActive — Check whether selection has nonzero range

Signature: bool TextEditor::selectionActive()

Calls getSelectionRange() with temporary output variables and returns its result. A selection flag alone is not enough: a zero-length anchor-to-cursor range is treated as inactive.

```
bool TextEditor::selectionActive()
{
        int a, b, c, d;
        return getSelectionRange(a, b, c, d);
    }
```

TextEditor::beginSelection — Set selection anchor

Signature: void TextEditor::beginSelection()

If selection has not already begun, marks it active and records the current line and column as the anchor. Keeping the original anchor lets subsequent Shift+arrow movements extend or shrink the selection.

```
void TextEditor::beginSelection()
{
        if (!hasSelection)
        {
            hasSelection = true;
            anchorLine = currentLineNumber;
            anchorCol = getCurrentColumn();
        }
    }
```

TextEditor::clearSelection — Deactivate selection

Signature: void TextEditor::clearSelection()

Sets hasSelection to false. It does not change the cursor or document text.

```
void TextEditor::clearSelection()

{

        hasSelection = false;

    }
```

TextEditor::selectAll — Select entire document

Signature: void TextEditor::selectAll()

Sets the anchor to line 0, column 0, then moves the cursor to INT_MAX/INT_MAX. moveTo() clamps this to the last line's end, making the range span the whole document.

```
void TextEditor::selectAll()

{

        hasSelection = true;

        anchorLine = 0;

        anchorCol = 0;



        moveTo(INT_MAX, INT_MAX);   // clamps to last line, end of line

    }
```

TextEditor::textBetween — Extract a range of text

Signature: string TextEditor::textBetween(int sl, int sc, int el, int ec)

Starts at the requested start line and walks through the inclusive line range. It copies characters from the start column through (but excluding) the end column, inserting newline separators between selected lines. It assumes endpoints are ordered and valid, as supplied by getSelectionRange().

```
string TextEditor::textBetween(int sl, int sc, int el, int ec)

{

        string result;



        LineNode* line = firstLine;

        int n = 0;



        while (line != NULL && n < sl)

        {

            line = line->next;

            n++;

        }



        for (; line != NULL && n <= el; n++, line = line->next)

        {
```

```
            int from = (n == sl) ? sc : 0;

            int to = (n == el) ? ec : INT_MAX;


            int col = 0;


            for (CharNode* c = line->head; c != NULL; c = c->next, col++)

            {

                if (col >= to)

                    break;


                if (col >= from)

                    result += c->data;

            }


            if (n < el)

                result += '\n';

        }


        return result;

    }
```

TextEditor::deleteSelection — Delete selected text

Signature: bool TextEditor::deleteSelection()

Gets the normalized selection range; if no non-empty range exists, clears selection and returns false. Otherwise it counts the selected string's characters including newline separators, moves to the start, calls deleteChar() that many times (which merges lines at end-of-line), clears selection, and returns true.

Range note: textBetween() counts newline separators too. deleteChar() merges adjacent lines when it reaches the end of a line, so the count includes those line breaks.

```
bool TextEditor::deleteSelection()

{

        int sl, sc, el, ec;


        if (!getSelectionRange(sl, sc, el, ec))

        {

            clearSelection();

            return false;

        }


        int count = (int)textBetween(sl, sc, el, ec).size();
```

```
        moveTo(sl, sc);

        // deleteChar() at end-of-line joins the next line,
        // so every character and every '\n' costs one call.
        for (int i = 0; i < count; i++)
            deleteChar();

        clearSelection();

        return true;
    }
```

TextEditor::setSystemClipboard — Write Windows system clipboard

Signature: void TextEditor::setSystemClipboard(const string& text)

Converts LF newlines to Windows CRLF, opens the system clipboard, empties it, allocates movable global memory, copies a null-terminated CF_TEXT string into it, and transfers ownership with SetClipboardData. It closes the clipboard on completion and frees memory if transfer fails.

Ownership note: when SetClipboardData succeeds, Windows owns the global-memory block; the code frees it only when transferring ownership fails.

```
void TextEditor::setSystemClipboard(const string& text)
{
        string out;

        for (size_t i = 0; i < text.size(); i++)
        {
            if (text[i] == '\n')
                out += "\r\n";
            else
                out += text[i];
        }

        if (!OpenClipboard(NULL))
            return;

        EmptyClipboard();

        HGLOBAL mem = GlobalAlloc(GMEM_MOVEABLE, out.size() + 1);

        if (mem != NULL)
```

```
        {
            char* dst = (char*)GlobalLock(mem);

            memcpy(dst, out.c_str(), out.size() + 1);

            GlobalUnlock(mem);

            if (SetClipboardData(CF_TEXT, mem) == NULL)
                GlobalFree(mem);
        }

        CloseClipboard();
    }
```

TextEditor::getSystemClipboard — Read Windows system clipboard

Signature: bool TextEditor::getSystemClipboard(string& text)

Opens the clipboard and retrieves CF_TEXT data. It locks the memory, copies the text into the output string while discarding carriage returns (normalizing CRLF to LF), unlocks the memory, closes the clipboard, and returns success/failure.

```
bool TextEditor::getSystemClipboard(string& text)
{
        if (!OpenClipboard(NULL))
            return false;

        HANDLE h = GetClipboardData(CF_TEXT);

        if (h == NULL)
        {
            CloseClipboard();
            return false;
        }

        const char* src = (const char*)GlobalLock(h);

        if (src == NULL)
        {
            CloseClipboard();
            return false;
        }
```

```
        text.clear();


        for (; *src; src++)
        {
            if (*src == '\r')
                continue;               // "\r\n" -> "\n"


            text += *src;
        }


        GlobalUnlock(h);
        CloseClipboard();


        return true;
    }
```

TextEditor::copy — Copy selected text

Signature: void TextEditor::copy()

Does nothing without a non-empty selection. Otherwise extracts selected text, stores it in the editor's clipboard string, builds an internal linked-list representation, and writes the text to the Windows system clipboard.

```
void TextEditor::copy()
{
        int sl, sc, el, ec;


        if (!getSelectionRange(sl, sc, el, ec))
            return;


        clipboard = textBetween(sl, sc, el, ec);


        buildClipList(clipboard);           // text -> linked list


        setSystemClipboard(clipboard);
    }
```

TextEditor::cut — Cut selected text

Signature: void TextEditor::cut()

Does nothing if no active non-empty selection exists. Otherwise copies the selection to the internal/system clipboard and then deletes the selection from the document.

```
void TextEditor::cut()
```

```
{
        if (!selectionActive())
            return;

        copy();
        deleteSelection();
    }
```

TextEditor::paste — Paste clipboard contents

Signature: void TextEditor::paste()

Reads the Windows system clipboard when available. If its text differs from the editor's cached clipboard, it rebuilds the internal clipboard list. It ignores an empty clipboard, deletes any active selection, and then inserts the internal clipboard representation at the cursor.

```
void TextEditor::paste()
{
        string text;

        bool haveSystem = getSystemClipboard(text) && !text.empty();

        // Reuse the prebuilt list; only rebuild if another app changed
        // the system clipboard since our last copy/cut.
        if (haveSystem && text != clipboard)
        {
            clipboard = text;
            buildClipList(text);
        }

        if (clipHead == NULL ||
            (clipLineCount == 1 && clipHead->head == NULL))
            return;

        // Pasting over a selection replaces it
        deleteSelection();

        pasteClipList();
    }
```

9. Keyboard event loop

TextEditor::start — Main input/event loop

Signature: void TextEditor::start()

Configures Windows console input and cursor settings, installs Ctrl+C handling, repeatedly redraws when needed, reads console events, and dispatches keyboard shortcuts and editing commands. It handles undo/redo, Escape, save/open, copy/cut/paste/select-all, arrow keys with Shift selection, Backspace/Delete, Enter, and printable ASCII. It captures a pre-edit snapshot and records edits after ordinary editing actions. On exit it restores the console handler/mode settings.

Control-flow note: Ctrl+Z and Ctrl+Y use continue, so the loop redraws on the next iteration without passing through recordEdit() for that keypress.

```
void TextEditor::start()

{

    HANDLE hInput =

        GetStdHandle(STD_INPUT_HANDLE);



    DWORD mode;



    GetConsoleMode(

        hInput,

        &mode

    );



    // No ENABLE_PROCESSED_INPUT: Ctrl+C arrives as a normal key event.

    SetConsoleMode(hInput, ENABLE_EXTENDED_FLAGS | ENABLE_WINDOW_INPUT );



    // Belt and braces: if the terminal still raises a Ctrl+C signal,

    // swallow it instead of letting it kill the program.

    SetConsoleCtrlHandler(NULL, TRUE);          // ignore Ctrl+C outright

    SetConsoleCtrlHandler(ctrlHandler, TRUE);



    HANDLE hOutput = GetStdHandle(STD_OUTPUT_HANDLE);



    CONSOLE_CURSOR_INFO cursorInfo;



    cursorInfo.dwSize = 20;

    cursorInfo.bVisible = TRUE;



    SetConsoleCursorInfo (hOutput, &cursorInfo);
```

```
INPUT_RECORD input;

DWORD events;

bool dirty = true;

while (true)
{
    if (dirty)
    {
        display();
        dirty = false;
    }

    ReadConsoleInput(
        hInput,
        &input,
        1,
        &events
    );

    if (input.EventType != KEY_EVENT)
        continue;

    KEY_EVENT_RECORD key =
        input.Event.KeyEvent;

    if (!key.bKeyDown)
        continue;

    dirty = true;       // redraw only after real key presses

    bool ctrl =
        (key.dwControlKeyState &
            (LEFT_CTRL_PRESSED | RIGHT_CTRL_PRESSED)) != 0;
```

```
bool shift =
    (key.dwControlKeyState & SHIFT_PRESSED) != 0;

WORD vk = key.wVirtualKeyCode;

EditorSnapshot beforeEdit = captureSnapshot();
bool isTyping = false;

if (ctrl && vk == 'Z')
{
    undo();
    continue;
}
else if (ctrl && vk == 'Y')
{
    redo();
    continue;
}

if (vk == VK_ESCAPE)
{
    break;
}



else if (ctrl && shift && vk == 'S')
{
    // Ctrl + Shift + S: Save As
    saveDocument(true);
}
else if (ctrl && vk == 'S')
{
    // Ctrl + S: Save current file
    saveDocument(false);
}
else if (ctrl && vk == 'O')
{
```

```
    // Ctrl + O: Open a specific file
    openDocument();
}

// Ctrl+C is also matched by its control character (ETX, 3) in
// case the terminal reports a different virtual-key code.
else if ((ctrl && vk == 'C') || key.uChar.AsciiChar == 3 ||
            (ctrl && vk == VK_INSERT))
{
    copy();
}
else if ((ctrl && vk == 'X') || (shift && vk == VK_DELETE))
{
    cut();
}
else if ((ctrl && vk == 'V') || (shift && vk == VK_INSERT))
{
    paste();
}
else if (ctrl && vk == 'A')
{
    selectAll();
}
else if (vk == VK_LEFT || vk == VK_RIGHT ||
            vk == VK_UP || vk == VK_DOWN)
{
    int sl, sc, el, ec;

    if (shift)
    {
        beginSelection();
    }
    else if (getSelectionRange(sl, sc, el, ec))
    {
        // Left/Right collapse the selection to its edge
        if (vk == VK_LEFT)
        {
```

```
            clearSelection();
            moveTo(sl, sc);
            continue;
        }
        else if (vk == VK_RIGHT)
        {
            clearSelection();
            moveTo(el, ec);
            continue;
        }

        clearSelection();
    }
    else
    {
        clearSelection();
    }

    if (vk == VK_LEFT)
        moveLeft();
    else if (vk == VK_RIGHT)
        moveRight();
    else if (vk == VK_UP)
        moveUp();
    else
        moveDown();
}
else if (vk == VK_BACK)
{
    if (!deleteSelection())
        backspace();
}
else if (vk == VK_DELETE)
{
    if (!deleteSelection())
        deleteChar();
}
```

```
        else if (vk == VK_RETURN)
        {
            deleteSelection();
            newLine();
        }
        else if (!ctrl && key.uChar.AsciiChar >= 32 && key.uChar.AsciiChar <= 126)
        {
            deleteSelection();
            insert(key.uChar.AsciiChar);
            isTyping = true;
        }
        recordEdit( beforeEdit, isTyping, isTyping && key.uChar.AsciiChar == ' ');
    }

    SetConsoleCtrlHandler(ctrlHandler, FALSE);
    SetConsoleCtrlHandler(NULL, FALSE);
    SetConsoleMode(hInput, mode);
}
```

10. Rendering and display

TextEditor::flushBuffer — Write display buffer

Signature: void TextEditor::flushBuffer(HANDLE hOut, string& buf)

If the buffer is non-empty, writes it to the console in one WriteConsoleA call and then clears the string. This reduces the number of console output calls during rendering.

```
void TextEditor::flushBuffer(HANDLE hOut, string& buf)
{
    if (buf.empty())
        return;

    DWORD written;

    WriteConsoleA(hOut, buf.c_str(), (DWORD)buf.size(), &written, NULL);

    buf.clear();
}
```

TextEditor::clearScreen — Clear console screen buffer

Signature: void TextEditor::clearScreen(HANDLE hOut)

Reads console dimensions and current text attributes, fills the console buffer with spaces and the existing attributes, then returns the console cursor to coordinate (0,0). If console information cannot be read, it returns without changes.

```
void TextEditor::clearScreen(HANDLE hOut)

{

        CONSOLE_SCREEN_BUFFER_INFO csbi;



        if (!GetConsoleScreenBufferInfo(hOut, &csbi))

            return;



        DWORD cells = (DWORD)csbi.dwSize.X * (DWORD)csbi.dwSize.Y;

        DWORD written;

        COORD home = { 0, 0 };



        FillConsoleOutputCharacterA(hOut, ' ', cells, home, &written);

        FillConsoleOutputAttribute(hOut, csbi.wAttributes, cells, home, &written);

        SetConsoleCursorPosition(hOut, home);

    }
```

TextEditor::display — Render document and selection

Signature: void TextEditor::display()

Clears the screen, computes the active selection range, and traverses every line and character. It batches runs of text sharing the same console colour into a string buffer, uses a different background attribute for selected characters, prints line breaks, flushes the final buffer, and positions the visible console cursor at the editor's logical column and line.

```
void TextEditor::display()

{

        HANDLE hOut = GetStdHandle(STD_OUTPUT_HANDLE);



        clearScreen(hOut);



        const WORD normalAttr =

            FOREGROUND_RED | FOREGROUND_GREEN | FOREGROUND_BLUE;



        const WORD selectedAttr =

            BACKGROUND_RED | BACKGROUND_GREEN | BACKGROUND_BLUE;



        int sl = 0, sc = 0, el = 0, ec = 0;

        bool selecting = getSelectionRange(sl, sc, el, ec);



        // Characters are collected into a buffer and written in one call

        // per run of identical colour, instead of one cout per character.
```

```
string buf;
WORD curAttr = normalAttr;

SetConsoleTextAttribute(hOut, curAttr);

LineNode* line =
    firstLine;

int row = 0;

while (line != NULL)
{
    CharNode* temp =
        line->head;

    int col = 0;

    while (temp != NULL)
    {
        bool selected = false;

        if (selecting)
        {
            bool afterStart =
                row > sl || (row == sl && col >= sc);

            bool beforeEnd =
                row < el || (row == el && col < ec);

            selected = afterStart && beforeEnd;
        }

        WORD attr = selected ? selectedAttr : normalAttr;

        if (attr != curAttr)
        {
            flushBuffer(hOut, buf);
```

```
                SetConsoleTextAttribute(hOut, attr);
                curAttr = attr;
            }

            buf += temp->data;

            temp = temp->next;
            col++;
        }

        if (curAttr != normalAttr)
        {
            flushBuffer(hOut, buf);
            SetConsoleTextAttribute(hOut, normalAttr);
            curAttr = normalAttr;
        }

        buf += '\n';

        line = line->next;
        row++;
    }

    flushBuffer(hOut, buf);

    int x = getCurrentColumn();
    int y = currentLineNumber;

    gotoxy(x, y);
}
```

TextEditor::showList — Print document contents

Signature: void TextEditor::showList()

Traverses the line list and character list, prints every character, then prints a newline for each line. It is a straightforward display/debug function and does not apply selection highlighting.

```
void TextEditor::showList()
{
    LineNode* line =
```

```
            firstLine;


        while (line != NULL)
        {
            CharNode* temp =
                line->head;


            while (temp != NULL)
            {
                cout << temp->data;


                temp = temp->next;
            }


            cout << endl;


            line = line->next;
        }
    }
```

11. Undo/redo data types and interface

This header declares EditorSnapshot and the Command interface; see command.cpp for the function bodies.

12. Undo/redo implementation

Command::record — Record undo state

Signature: void Command::record(const EditorSnapshot& before, const EditorSnapshot& after)

Ignores edits whose before/after text is identical. Otherwise pushes the pre-edit snapshot onto the undo stack and clears the redo stack, because a new edit invalidates the old redo path. Cursor-only changes are not recorded when the text is unchanged.

```
void Command::record(const EditorSnapshot& before, const EditorSnapshot& after)
{
    if (before.text == after.text)
        return;

    undoStack.push(before);
    while (!redoStack.empty())
        redoStack.pop();
}
```

Command::undo — Pop undo snapshot

Signature: bool Command::undo(const EditorSnapshot& current, EditorSnapshot& target)

Returns false if the undo stack is empty. Otherwise pushes the current snapshot onto the redo stack, copies the latest undo snapshot into the output target, pops it, and returns true.

```
bool Command::undo(const EditorSnapshot& current, EditorSnapshot& target)
{
    if (undoStack.empty())
        return false;

    redoStack.push(current);
    target = undoStack.top();
    undoStack.pop();
    return true;
}
```

Command::redo — Pop redo snapshot

Signature: bool Command::redo(const EditorSnapshot& current, EditorSnapshot& target)

Returns false if the redo stack is empty. Otherwise pushes the current snapshot onto the undo stack, copies the latest redo snapshot into the output target, pops it, and returns true.

```
bool Command::redo(const EditorSnapshot& current, EditorSnapshot& target)
{
    if (redoStack.empty())
        return false;

    undoStack.push(current);
    target = redoStack.top();
    redoStack.pop();
    return true;
}
```

Command::clear — Clear history stacks

Signature: void Command::clear()

Pops all entries from both undo and redo stacks. It is available as a history-reset helper, though it is not called by the visible editor flow in this source version.

```
void Command::clear()
{
    while (!undoStack.empty()) undoStack.pop();
    while (!redoStack.empty()) redoStack.pop();
}
```

13. File I/O interface

This header declares saveToFile() and loadFromFile(); their behaviour is described in FileHandler.cpp.

14. File I/O implementation

saveToFile — Write content to a file

Signature: bool saveToFile(const std::string& filename, const std::string& content)

Opens the target path in binary output mode, writes the exact bytes from the supplied string, checks stream status, closes the file, and returns whether the write/close succeeded.

```
bool saveToFile(const std::string& filename, const std::string& content)

{

    std::ofstream file(filename.c_str(),std::ios::binary);



    if (!file.is_open())

        return false;
```

```
    file.write(content.data(),static_cast<std::streamsize>(content.size()));


    if (!file.good())

        return false;


    file.close();

    return !file.fail();

}
```

loadFromFile — Read file and normalize line endings

Signature: bool loadFromFile(const std::string& filename,std::string& content)

Opens the file in binary mode, reads all bytes into a temporary string, rejects stream read errors, then removes carriage returns that precede line feeds so CRLF becomes LF. The normalized content is returned through the output reference.

```
bool loadFromFile(const std::string& filename,std::string& content)

{

    std::ifstream file(filename.c_str(), std::ios::binary);


    if (!file.is_open())

        return false;


    std::string temp(

        (std::istreambuf_iterator<char>(file)),

        std::istreambuf_iterator<char>()

    );


    if (file.bad())

        return false;


    // Convert Windows CRLF line endings to LF.

    std::string normalized;

    normalized.reserve(temp.size());


    for (std::size_t i = 0; i < temp.size(); ++i)

    {

        if (temp[i] == '\r' &&

            i + 1 < temp.size() &&

            temp[i + 1] == '\n')

        {

            continue;
```

```
        }

        normalized += temp[i];
    }

    content = normalized;
    return true;
}
```

15. Console helper interface

This header declares gotoxy() and ctrlHandler(); their behaviour is described in ConsoleHelpers.cpp.

16. Console helper implementation

gotoxy — Set visible console cursor position

Signature: void gotoxy(int x, int y)

Builds a Windows COORD from x/y and calls SetConsoleCursorPosition on standard output. Used after rendering to align the visible cursor with the editor's logical cursor.

```
void gotoxy(int x, int y)
{
    COORD pos;
    pos.X = x;
    pos.Y = y;

    SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE),pos);
}
```

ctrlHandler — Handle Ctrl+C console event

Signature: BOOL WINAPI ctrlHandler(DWORD type)

Returns true only for CTRL_C_EVENT, telling Windows that this event was handled. This supports using Ctrl+C as the editor's copy shortcut rather than terminating the program.

```
BOOL WINAPI ctrlHandler(DWORD type)
{
    return type == CTRL_C_EVENT;
}
```

Part III — End-to-end keyboard flow

The start() function is the dispatcher. For each key-down event it first identifies Ctrl/Shift modifiers, captures a pre-edit snapshot, then chooses the matching command.

- Ctrl+Z / Ctrl+Y: call undo() / redo() and immediately continue the loop.
- Escape: exit the editor loop.
- Ctrl+S / Ctrl+Shift+S / Ctrl+O: save, Save As, or open.
- Ctrl+C / Ctrl+X / Ctrl+V / Ctrl+A: copy, cut, paste, or select all.
- Arrow keys: move cursor; holding Shift begins/extends a selection. Left/right collapse an existing selection to its nearer edge when Shift is not held.
- Backspace/Delete: delete the selection if present; otherwise delete one character or join lines at a boundary.
- Enter: delete any active selection, then split the current line.
- Printable ASCII: replace selection if any, then insert the typed character and mark it as typing for history grouping.
- After ordinary actions, recordEdit() compares before/after states through Command::record().

Part IV — Important implementation notes

Cursor representation: cursor is not an integer. It points to the character immediately to the right of the insertion point; NULL means the insertion point is after the line's tail.

Zero-based positions: Line numbers and columns begin at zero internally. Display coordinates use the same logical row/column convention.

Undo/redo snapshots: The history stores whole-document snapshots, not inverse linked-list operations. Undo/redo restore text using setText() and then restore the cursor.

Typing grouping: The current grouping logic records the first pre-edit state when a typing group opens, and closes the group after a space or a non-typing edit. This is simpler than a full command-per-action history.

Skip list: The skip list is rebuilt after structural line changes. The builder uses deterministic skip intervals rather than random tower heights.

Text character support: setText() and buildClipList() keep printable ASCII characters, expand tabs to four spaces, and treat newline as a line boundary. This is not a full Unicode text model.

Destructor cleanup: The visible destructor frees the internal clipboard list but does not visibly free the main document list or skip-list nodes. This is a potential resource leak to address later.

Console portability: The project uses windows.h and Windows console/clipboard APIs, so it is intended for Windows rather than a portable terminal environment.

Potential history detail: Command::record() compares text only. It does not record cursor-only movement because unchanged text returns early.

Suggested study order

1. Read EditorNodes.h until the pointer relationships are clear.
2. Read TextEditor.h to learn the class state and method names.
3. Trace insert(), backspace(), deleteChar(), newLine(), and the two merge functions on paper.
4. Study Cursor.cpp and practise translating cursor pointer states into columns.
5. Study getText()/setText() before studying undo/redo, because snapshots use them.
6. Study selection and clipboard helpers, then multi-line paste.
7. Finish with start() and display(), which connect keyboard input and rendering to the lower-level operations.

Addendum — Destructor function reference

TextEditor::~TextEditor() — Destructor

Exact role: this function runs automatically when a TextEditor object is destroyed. Its body calls clearClipList(), which deletes the internal clipboard's character nodes and line nodes and resets the clipboard pointers/count.

Important limitation in this source version: the destructor does not visibly call clearSkipList() or free the main document's LineNode and CharNode chains. That may leave dynamically allocated memory unreleased when the program exits. We can inspect and improve cleanup later, separately from understanding the existing implementation.

```
TextEditor::~TextEditor()
{
    clearClipList();
}
```